

Diferencias entre MySQL, PostgreSQL y SQLite

Tipos de datos más usados

 **Caution**

MySQL no contempla mayúscula ni minúsculas en los nombres de sus columnas. Pero **PostgreSQL** y **SQLite** Sí

Propósito	MySQL	PostgreSQL	SQLite
Enteros pequeños	TINYINT (1 byte, -128 a 127)	SMALLINT (2 bytes, -32,768 a 32,767)	INTEGER (mapeado dinámicamente)
Enteros medianos	INT o INTEGER (4 bytes)	INTEGER (4 bytes)	INTEGER (sin tamaño fijo)
Enteros grandes	BIGINT (8 bytes)	BIGINT (8 bytes)	INTEGER (mapeado dinámicamente)
Autoincrementales	INT AUTO_INCREMENT	SERIAL o BIGSERIAL	INTEGER PRIMARY KEY AUTOINCREMENT
Números decimales	DECIMAL(m, d) o NUMERIC	NUMERIC(m, d)	REAL o NUMERIC
Números en coma flotante	FLOAT (4 bytes), DOUBLE (8 bytes)	REAL (8 bytes)	REAL
Texto corto	VARCHAR(n) (hasta 65,535)	VARCHAR(n) (longitud definida)	TEXT (sin longitud definida)
Texto largo	TEXT (largo ilimitado)	TEXT	TEXT
Datos binarios	BLOB	BYTEA	BLOB
Fecha	DATE	DATE	TEXT (ISO 8601)
Fecha y hora	DATETIME o TIMESTAMP	TIMESTAMP con zona horaria	TEXT (ISO 8601)

Propósito	MySQL	PostgreSQL	SQLite
Hora	TIME	TIME	No hay tipo específico; usa TEXT
Booleanos	TINYINT(1) o BOOLEAN (alias)	BOOLEAN (TRUE/FALSE)	No hay tipo específico; usa INTEGER (0/1)
UUID	No nativo; usa CHAR(36)	UUID	No nativo; usa TEXT
JSON	JSON	JSON o JSONB	No nativo; usa TEXT
Enumerados	ENUM('val1', 'val2', ...)	No nativo; usa CHECK o tablas asociadas	No nativo; usa TEXT o tablas asociadas
Series/secuencias	No nativo; usa AUTO_INCREMENT	SERIAL o SEQUENCE	No soporta secuencias

Creación de ENUMS

1. ENUMs en MySQL:

1. ENUMs en PostgreSQL (soporte nativo)

PostgreSQL permite definir tipos ENUM personalizados directamente, lo que facilita su uso. Aquí tienes un ejemplo:

Crear un ENUM:

```
1 CREATE TYPE estado_pedido AS ENUM ('pendiente', 'en_proceso', 'completado');
```

Usarlo en una tabla:

```
1 CREATE TABLE pedidos (  
2     id SERIAL PRIMARY KEY,  
3     cliente TEXT NOT NULL,  
4     estado estado_pedido NOT NULL DEFAULT 'pendiente'  
5 );
```

Insertar datos:

```
1 INSERT INTO pedidos (cliente, estado) VALUES ('Juan Pérez', 'en_proceso');
```

2. ENUMs en SQLite (no soporte nativo)

SQLite no tiene soporte nativo para **ENUMs**, pero puedes lograr un comportamiento similar usando restricciones **CHECK** o tablas relacionadas:

Opción 1: Usar **CHECK** para restringir valores:

Puedes definir una restricción **CHECK** en una columna para validar los valores permitidos.

```
1 CREATE TABLE pedidos (  
2     id INTEGER PRIMARY KEY,  
3     cliente TEXT NOT NULL,  
4     estado TEXT NOT NULL CHECK (estado IN ('pendiente', 'en_proceso', 'completado'))  
5 );
```

Insertar datos:

```
1 INSERT INTO pedidos (cliente, estado) VALUES ('Juan Pérez', 'en_proceso'); -- Correcto
2 INSERT INTO pedidos (cliente, estado) VALUES ('Juan Pérez', 'cancelado'); -- Error: no permitido
```

Opción 2: Usar una tabla asociada para ENUMs:

Crea una tabla separada para los valores permitidos y utiliza una clave foránea para validarlos.

```
1 CREATE TABLE estado_pedido (
2     id INTEGER PRIMARY KEY,
3     estado TEXT UNIQUE
4 );
5
6 INSERT INTO estado_pedido (estado) VALUES ('pendiente'), ('en_proceso'), ('completado');
7
8 CREATE TABLE pedidos (
9     id INTEGER PRIMARY KEY,
10    cliente TEXT NOT NULL,
11    estado TEXT NOT NULL,
12    FOREIGN KEY (estado) REFERENCES estado_pedido(estado)
13 );
```

Insertar datos:

```
1 INSERT INTO pedidos (cliente, estado) VALUES ('Juan Pérez', 'en_proceso'); -- Correcto
2 INSERT INTO pedidos (cliente, estado) VALUES ('Juan Pérez', 'cancelado'); -- Error: no permitido
```

Diferencias entre Sentencias SQL

Operación	MySQL	PostgreSQL	SQLite
Creación de tablas si no existe	<code>CREATE TABLE IF NOT EXISTS table_name</code>	<code>CREATE TABLE IF NOT EXISTS table_name</code>	<code>CREATE TABLE IF NOT EXISTS table_name</code>
Modificar tipo de columna	<code>ALTER TABLE table_name MODIFY COLUMN col ...</code>	<code>ALTER TABLE table_name ALTER COLUMN col TYPE ...</code>	No se puede cambiar el tipo de columna directamente
Renombrar columna	<code>ALTER TABLE table_name CHANGE old_col new_col ...</code>	<code>ALTER TABLE table_name RENAME COLUMN old_col TO new_col</code>	<code>ALTER TABLE table_name RENAME COLUMN old_col TO new_col</code>
Agregar columna	<code>ALTER TABLE table_name ADD COLUMN col_name ...</code>	<code>ALTER TABLE table_name ADD COLUMN col_name ...</code>	<code>ALTER TABLE table_name ADD COLUMN col_name ...</code>
Eliminar columna	<code>ALTER TABLE table_name DROP COLUMN col_name</code>	<code>ALTER TABLE table_name DROP COLUMN col_name</code>	No soporta eliminación de columna
Insertar datos	<code>INSERT INTO table_name (col1, col2) VALUES ...</code>	Igual que MySQL	Igual que MySQL
Insertar datos si no existen	<code>INSERT IGNORE INTO table_name ...</code> o <code>REPLACE INTO</code>	<code>INSERT ... ON CONFLICT DO NOTHING</code>	<code>INSERT OR IGNORE INTO table_name ...</code>
Obtener fecha y hora actual	<code>NOW()</code>	<code>NOW()</code>	<code>datetime('now')</code> (sin paréntesis en SQL simple)
Comparar valores nulos	<code>IS NULL</code> y <code>IS NOT NULL</code>	<code>IS NULL</code> y <code>IS NOT NULL</code>	<code>IS NULL</code> y <code>IS NOT NULL</code>
Función de concatenación	<code>CONCAT(col1, col2)</code>	<code>CONCAT(col1, col2)</code> o <code>col1 col2</code>	<code>col1 col2</code>

Operación	MySQL	PostgreSQL	SQLite
Operador matemático para residuo	%	%	%
Subconsulta	Soportado	Soportado	Soportado
Función para obtener año de fecha	YEAR(fecha_col)	EXTRACT(YEAR FROM fecha_col)	strftime('%Y', fecha_col)
Eliminar filas con condición	DELETE FROM table_name WHERE condition	Igual que MySQL	Igual que MySQL
Eliminar todos los registros	DELETE FROM table_name o TRUNCATE	DELETE FROM table_name (sin TRUNCATE)	DELETE FROM table_name
Limitar número de filas en consulta	SELECT ... LIMIT n	SELECT ... LIMIT n	SELECT ... LIMIT n
Obtener filas de un rango específico	SELECT ... LIMIT n OFFSET m	SELECT ... LIMIT n OFFSET m	SELECT ... LIMIT n OFFSET m
Unir tablas	INNER JOIN , LEFT JOIN , RIGHT JOIN	INNER JOIN , LEFT JOIN , RIGHT JOIN , FULL JOIN	INNER JOIN , LEFT JOIN (sin RIGHT JOIN)
Índices	CREATE INDEX idx_name ON table_name(col_name)	Igual que MySQL	Igual que MySQL
Comentarios	-- comentario y /* comentario */	-- comentario y /* comentario */	-- comentario y /* comentario */